

Microservice Communication

1. Why Inter-Service Communication Is Needed

In a microservices architecture, each service:

- Owns its **own data**
- Is **independently deployable**
- Has a **single business responsibility**

Because business workflows span multiple services (for example, *Order* → *Payment* → *Inventory* → *Notification*), services **must communicate** to complete an end-to-end use case.

2. Two Fundamental Communication Models

A. Synchronous Communication

- Caller **waits** for the response
- Typically request–response
- Commonly HTTP-based

B. Asynchronous Communication

- Caller **does not wait**
- Event-driven
- Uses message brokers or queues

Both are used together in real systems.

3. Synchronous Communication (HTTP / REST / gRPC)

- One microservice **directly calls** another.
- The caller **waits** for the response.
- Typically implemented using **HTTP/REST**.

How It Works

1. Service A sends an HTTP request to Service B

2. Service B processes the request
3. Service B returns a response
4. Service A continues execution

Common Protocols

Protocol	Usage
REST (HTTP/JSON)	Most common, simple
gRPC	High-performance, binary, internal services
GraphQL	Aggregated reads

Example (REST)

OrderService → GET /api/products/{id} → ProductService

Advantages

- Simple to understand and implement
- Immediate response
- Easy debugging

Disadvantages

- **Tight coupling**
- Cascading failures
- Latency increases with call chains
- Requires resilience patterns

When to Use

- Querying data
- Validation checks
- Read-heavy operations

- Low-latency internal calls

4. Asynchronous Communication (Messaging / Events)

- Services **do not call each other directly**
- Events or messages are published to a **message broker (RabbitMQ, Service Bus)**
- Other services consume events **independently**

How It Works

1. Service A publishes an **event**
2. Message broker stores and delivers it
3. One or more services consume the event
4. Each consumer processes independently

Message Patterns

Pattern	Description
Event	Something happened
Command	Do something
Queue	One consumer
Topic	Multiple consumers

Example

OrderPlacedEvent

→ InventoryService

→ PaymentService

→ EmailService

Advantages

- Loose coupling

- High scalability
- Fault tolerant
- Supports eventual consistency

Disadvantages

- More complex debugging
- Eventual consistency
- Requires idempotency handling

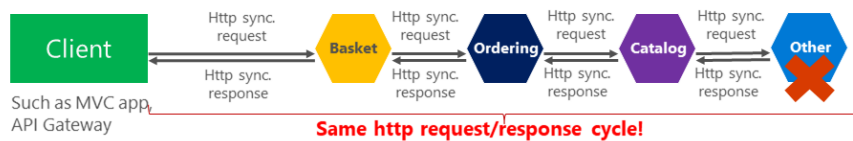
When to Use

- Business workflows
- Notifications
- Long-running processes
- Integration with multiple services

Synchronous vs. async communication across microservices

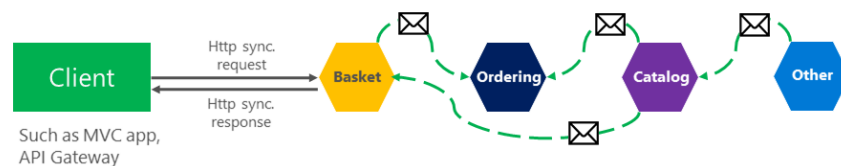
Anti-pattern

Synchronous
all request/response cycle



Asynchronous

Comm. across internal microservices
(EventBus: like **AMQP**)



"Asynchronous"

Comm. across internal microservices
(Polling: **Http**)

